

Incremental Indexing Checkpoint

Summary

This branch implements document-level incremental indexing for TokenSmith. The indexer now tracks source markdown files in a manifest, reuses unchanged per-document artifacts, and rebuilds only new or modified documents before merging the active document artifacts into the existing global retrieval outputs.

The implementation is on:

```
feature/incremental-indexing
```

What Changed

`src/index_builder.py` now supports both a single markdown file and a list of markdown files. For each markdown file, it:

1. Computes a SHA-256 source hash.
2. Computes a config fingerprint from the chunking config, embedding model path, heading mode, and table setting.
3. Checks `<index_prefix>_manifest.json` to determine whether the document can be reused.
4. Reuses saved per-document chunks, metadata, page map, and embeddings when the hash and config fingerprint match.
5. Rebuilds only new or changed documents.
6. Merges active document artifacts into the existing global files:

```
<index_prefix>.faiss  
<index_prefix>_bm25.pkl  
<index_prefix>_chunks.pkl  
<index_prefix>_sources.pkl  
<index_prefix>_meta.pkl  
<index_prefix>_page_to_chunk_map.json
```

`src/main.py` now indexes all `data/*.md` files by default instead of only the first markdown file. It also adds `-markdown_file` for controlled single-file runs:

```
python -m src.main index --index_prefix incremental_sample --markdown_file  
data/incremental_indexing_sample.md
```

`src/index_builder.py` also prints timing information for hash checks, artifact reuse, document rebuilds, model loading, embedding, FAISS rebuild, BM25 rebuild, and artifact writes.

`src/embedder.py` now uses `verbose=False` for llama.cpp model loading so benchmark output is readable.

Design Decisions

The implementation caches per-document chunks and embeddings, but still rebuilds the global FAISS and BM25 artifacts from active document artifacts.

This was intentional. It preserves compatibility with the existing retriever and `load_artifacts()` code, avoids FAISS delete/update complexity, and keeps BM25 correctness simple because BM25 corpus statistics depend on the full active corpus.

Document-level incremental indexing was chosen as the baseline because it matches the proposal goals. Section-level or chunk-level reuse remains future work because it would require stable chunk IDs, section hashing, and more careful page-map handling.

Deleted documents are excluded from the merged global index when they no longer appear in the current markdown file list. Their old per-document artifact files are not physically deleted yet.

Validation

Added:

```
tests/test_incremental_indexing.py
```

The tests use a fake chunker and fake embedder, so they do not require real GGUF models. They verify:

- unchanged documents are reused without re-embedding
- a modified document is rebuilt while unchanged documents are reused
- a deleted document is omitted from the merged global index
- `--markdown_file` indexes only the requested file instead of all `data/*.md` files

Focused test result:

```
4 passed, 3 warnings in 33.76s
```

The warnings are FAISS/SWIG deprecation warnings, not failures in the incremental indexing logic.

`tests/conftest.py` was also adjusted so missing optional HTML report dependencies, such as `google-genai`, do not fail an otherwise successful unit-test run.

Benchmark Result

A small controlled markdown sample was added at:

```
examples/incremental_indexing_sample.md
```

It can be copied into `data/` for a lightweight benchmark:

```
mkdir -p data
cp examples/incremental_indexing_sample.md data/incremental_indexing_sample.md
python -m src.main index --index_prefix incremental_sample --markdown_file
data/incremental_indexing_sample.md
python -m src.main index --index_prefix incremental_sample --markdown_file
data/incremental_indexing_sample.md
```

Observed sample timings:

```
first build:
total=190.894s
document_rebuild=190.528s
embedding=5.785s
faiss=0.109s
bm25=0.045s

unchanged incremental build:
total=0.162s
document_rebuild=0.000s
model_load=0.000s
embedding=0.000s
faiss=0.014s
bm25=0.023s
```

For this controlled sample, the unchanged incremental rebuild reused all 3 cached chunks and embedded 0 chunks. The global FAISS and BM25 rebuild cost on the unchanged run was only 0.037s combined.

This supports the current design choice for the baseline implementation: cache the expensive per-document artifacts and embeddings, while rebuilding global FAISS/BM25 for correctness and compatibility.

This should not be overstated as a full-corpus result. A larger end-to-end benchmark was not rerun because it would take too long. A safe writeup claim is:

```
Because a full-corpus benchmark was too costly to rerun within the project time
budget, I used a small controlled markdown document to validate the design. The
benchmark shows that unchanged documents do not reload the embedding model and do
not re-embed chunks. FAISS and BM25 are still rebuilt globally for correctness,
and in the controlled run their combined cost was 0.037s on the unchanged
incremental rebuild. This supports the current implementation while leaving full-
corpus FAISS/BM25 scaling as future evaluation work.
```

Commit Note

Avoid `git add .` because there are local generated/untracked files that should not be accidentally committed:

```
data/extracted_sections.json  
index/sections/textbook_index_page_to_chunk_map.json  
index/sections/incremental_sample_manifest.json  
index/sections/incremental_sample_page_to_chunk_map.json  
.codex
```

Use:

```
git add src/index_builder.py src/main.py src/embedder.py tests/conftest.py  
tests/test_incremental_indexing.py examples/incremental_indexing_sample.md  
checkpoint.md  
git commit -m "Add document-level incremental indexing"
```